

Introduction to Natural Language Processing

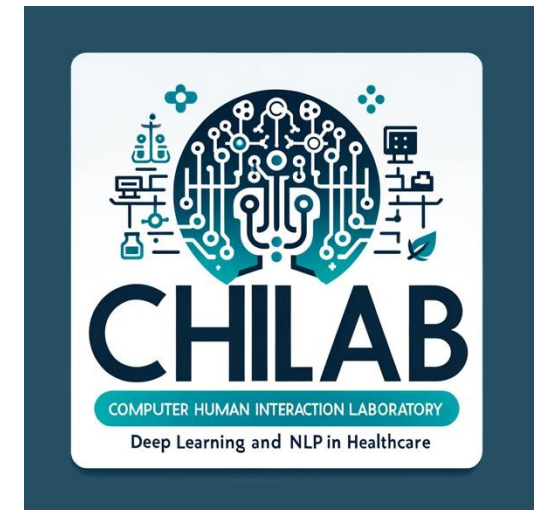
Irene Siragusa, PhD

Outline

-  Introduction
-  Natural Language Processing
-  Transformers
-  Attention Mechanism
-  Decoder-only models
-  Generation techniques
-  Prompt Engineering

Introduzione

- Irene Siragusa
 - Information and Communication Technologies (PhD)
 - Computer Science Engineering (MSc)
 - Computer Science and Telecommunication Engineering
- irene.siragusa02@unipa.it
- Computer-Human Interaction Laboratory (CHILab) @ University of Palermo (UniPa)
Palermo, Sicily, Italy



Introduzione

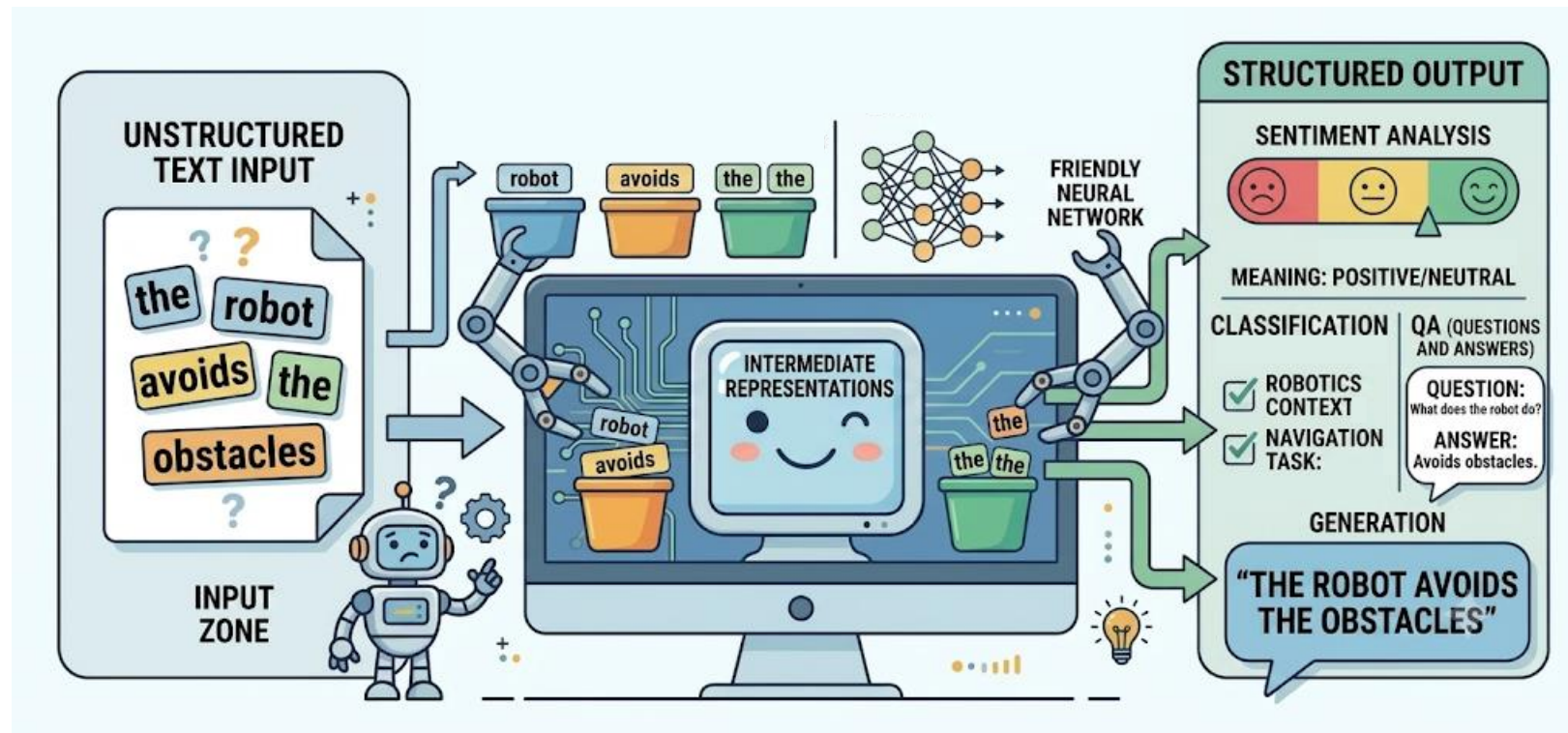
? *Subject material*

- NLP-EN-JP GitHub
 - Slides with theory
 - python notebooks with practice



<https://github.com/iresiragusa/NLP-EN-JP/>

Natural Language Processing



Natural Language Processing

- Natural Language Processing (NLP)
 - NLP is the use of human languages, such as English, Japanese or Italian, by a computer.
 - Many NLP applications are based on language models that define a probability distribution over sequences of words, characters, or bytes in a natural language.

NLP applications

Text pre-processing

Information Extraction

Machine translation

Text Reasoning

Summarization

Textual classification

Sentiment Analysis

Hate Speech Detection

Chatbots

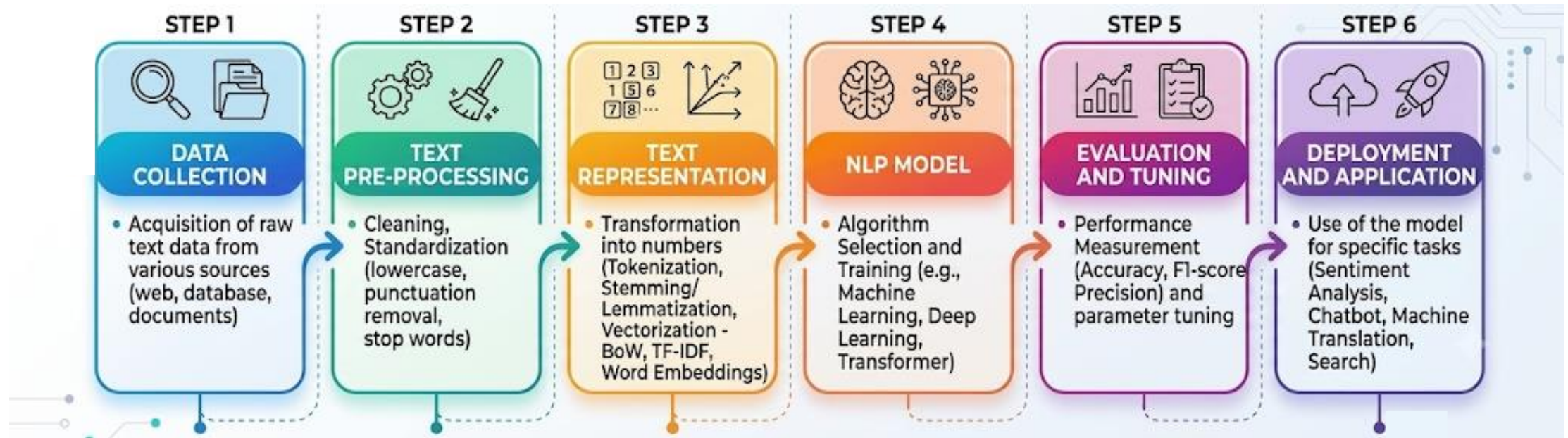
Text-to-Text Generation

Text-to-Image and viceversa



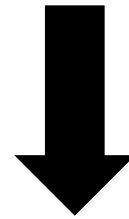
Image by NLPlanet 

NLP Pipeline



Tokenization

A misty ridge uprises from the surge



tokenization

['a', 'misty', 'ridge', 'up', '##rise', '##s', 'from', 'the', 'surge']

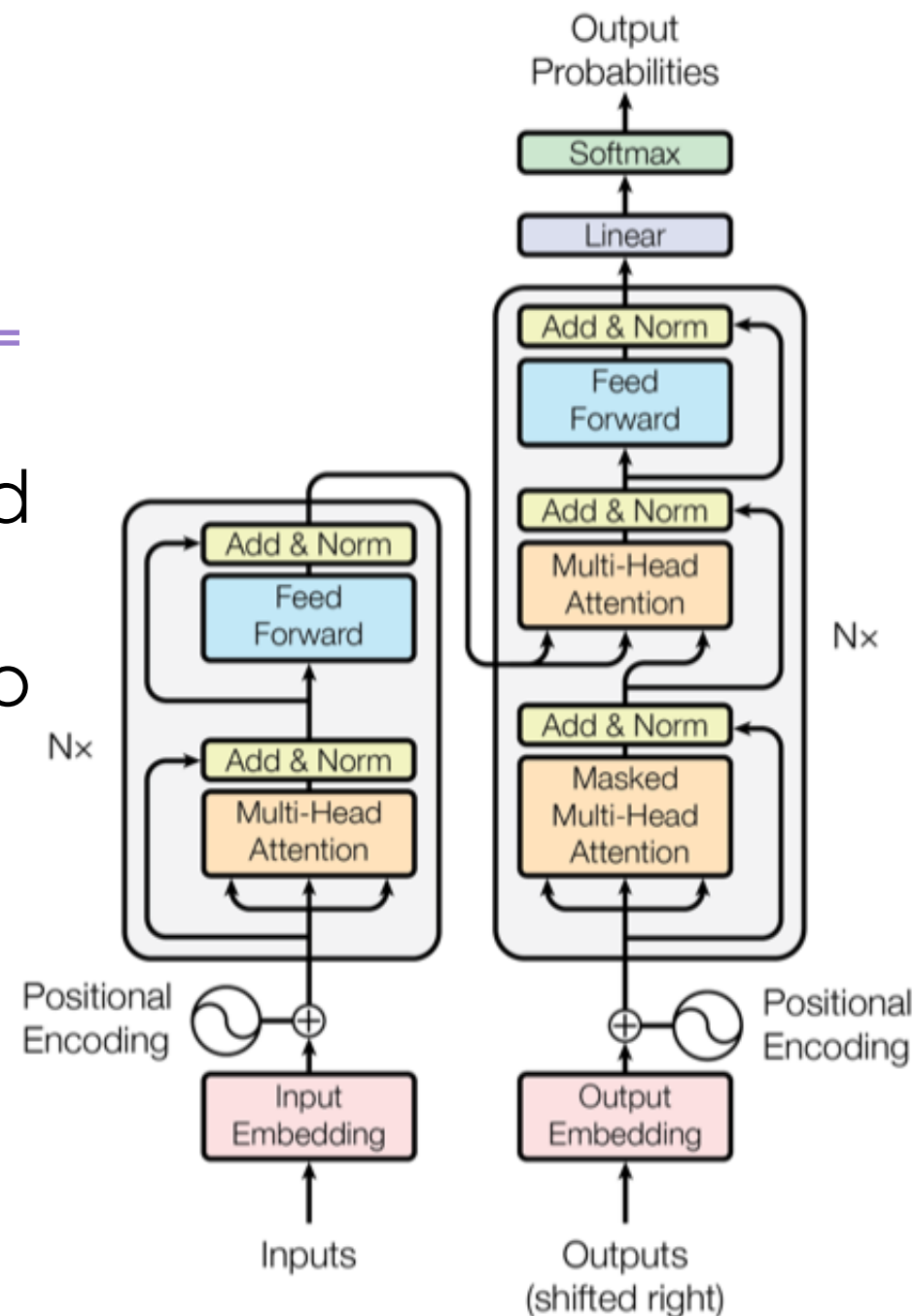
Large Language Models

- I Large Language Models (LLMs):
 - ✓ Assign a probability to given word sequences
 - ✓ Generates text by sampling the most probable next words
 - ✓ Are based on transformers architecture
 - 💡 *Are trained to predict the next correct word, thus they do not have a proper knowledge*

Transformers

- Transformers are a Neural Network architecture made up of encoder and decoder
- They relies on attention mechanisms to draw draw global dependencies between input and output

$$\begin{aligned}(\mathbf{z}_1, \dots, \mathbf{z}_n) &= \text{encoder}(\mathbf{x}_1, \dots, \mathbf{x}_n) \\ (\mathbf{y}_1, \dots, \mathbf{y}_m) &= \text{decoder}(\mathbf{z}_1, \dots, \mathbf{z}_n)\end{aligned}$$



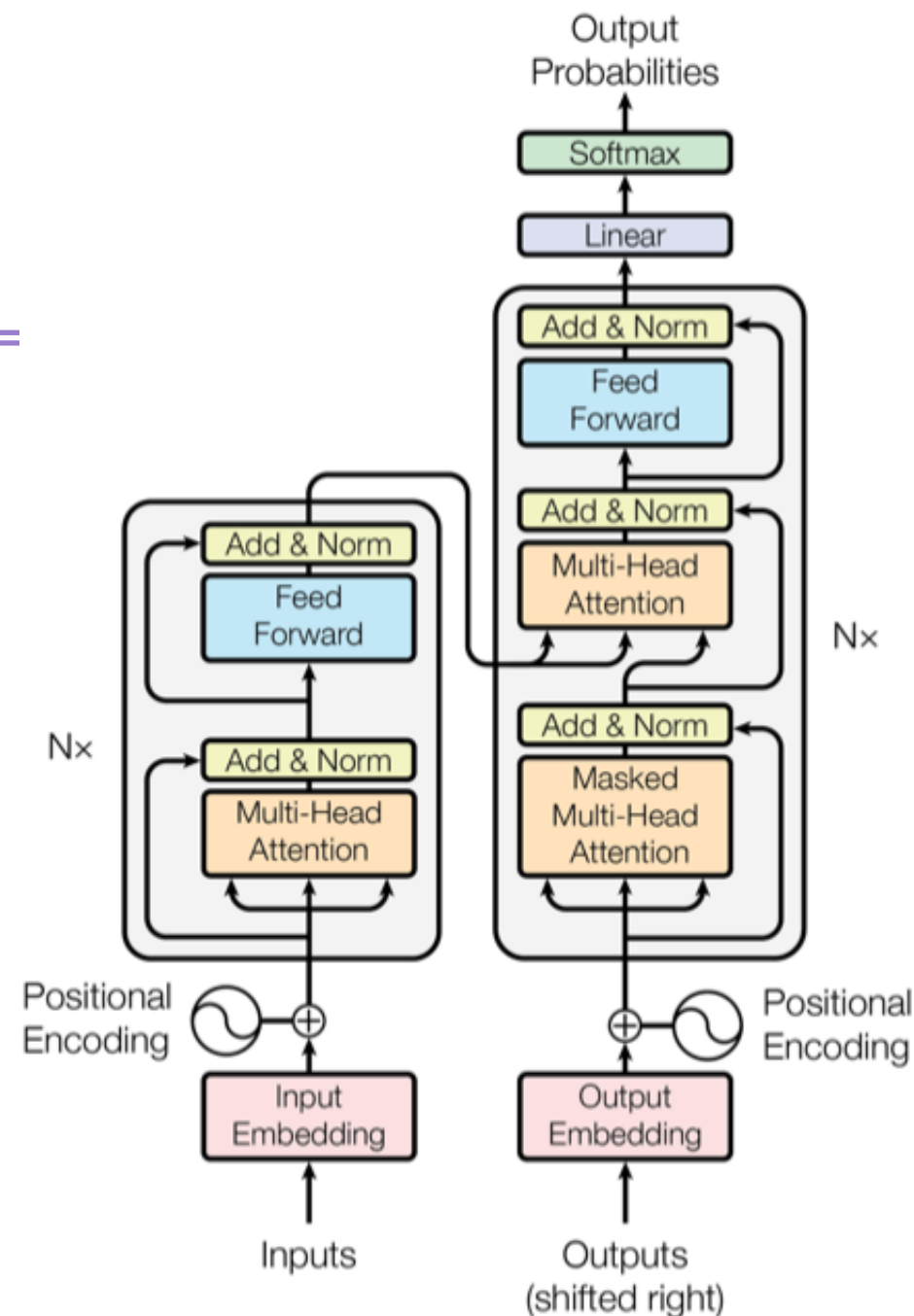
Transformers

- Encoder

- Multi-head self-attention mechanism + residual connection + normalization layer
- Feed-forward fully connected layer + residual connection + normalization layer

- Decoder

- Multi-head self-attention mechanism + residual connection + normalization layer
- Multi-head attention over the output of the encoder stack + residual connection + normalization layer
- Feed-forward fully connected layer + residual connection + normalization layer



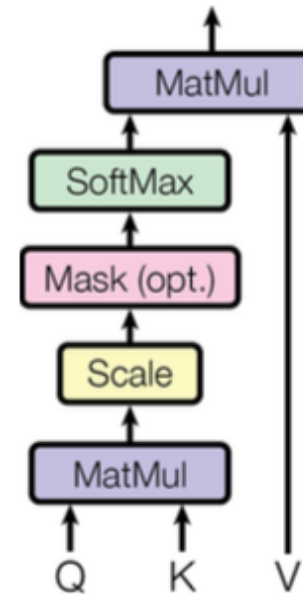
Transformers

From an input embedding three roles are played:

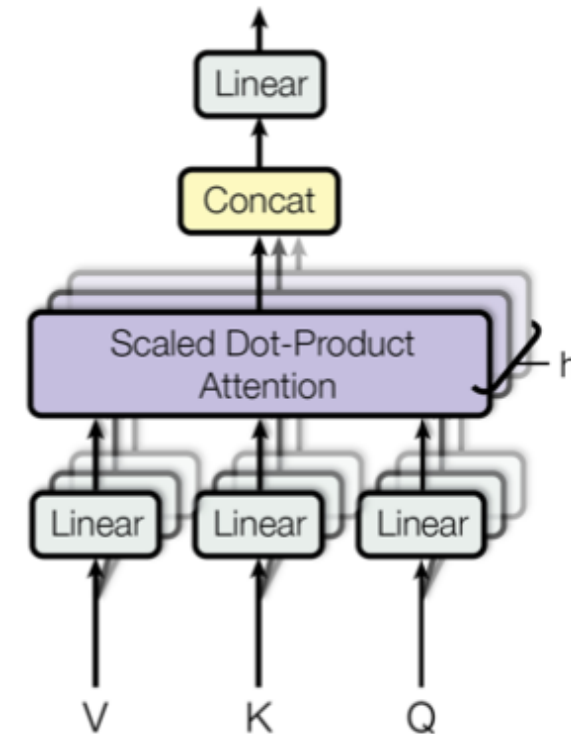
- Query → the current focus of attention
- Key → a preceding input
- Value → the value for the computation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Scaled Dot-Product Attention

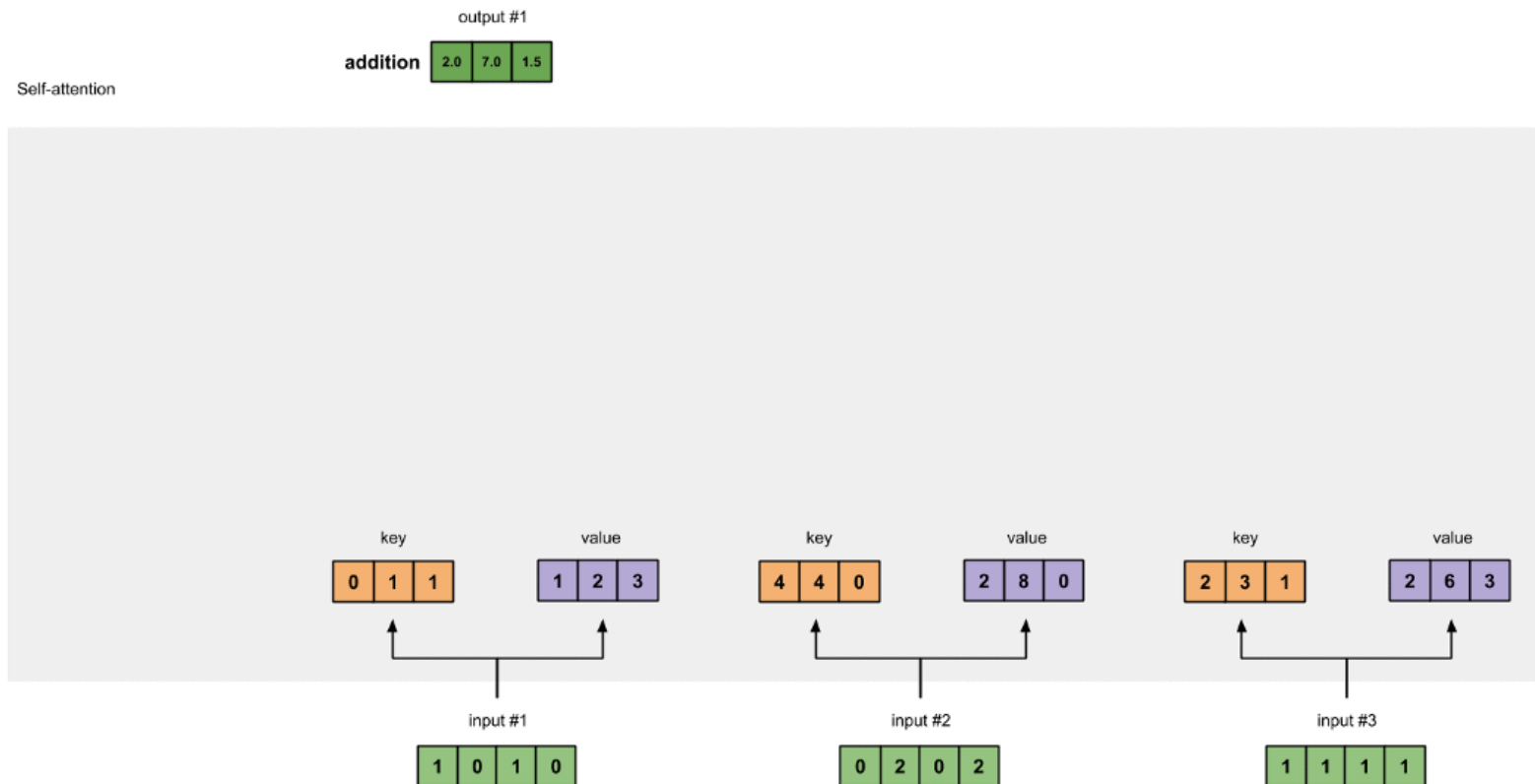


Multi-Head Attention



Transformers

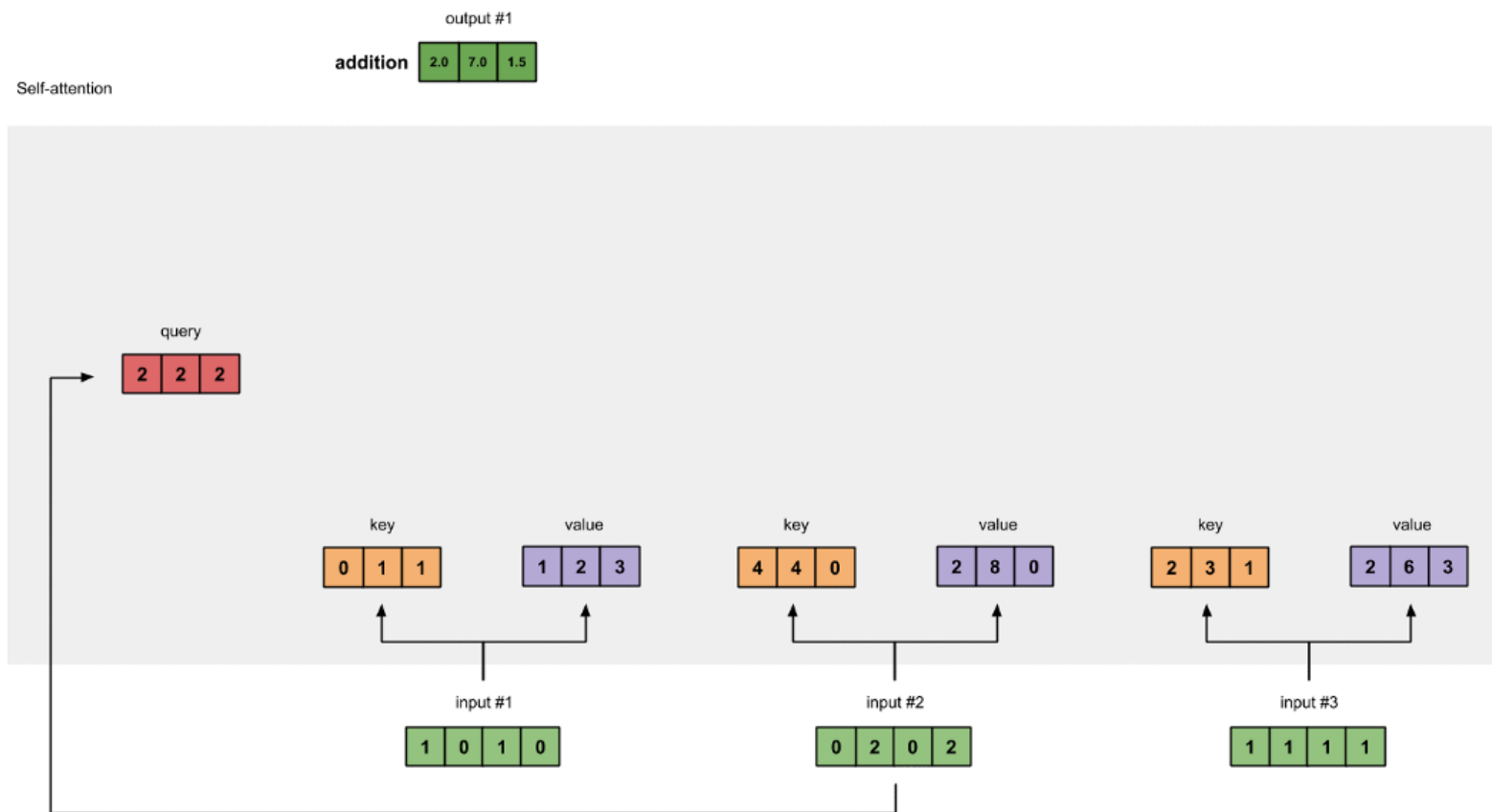
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



From a given
input,
calculate
Key, Value

Transformers

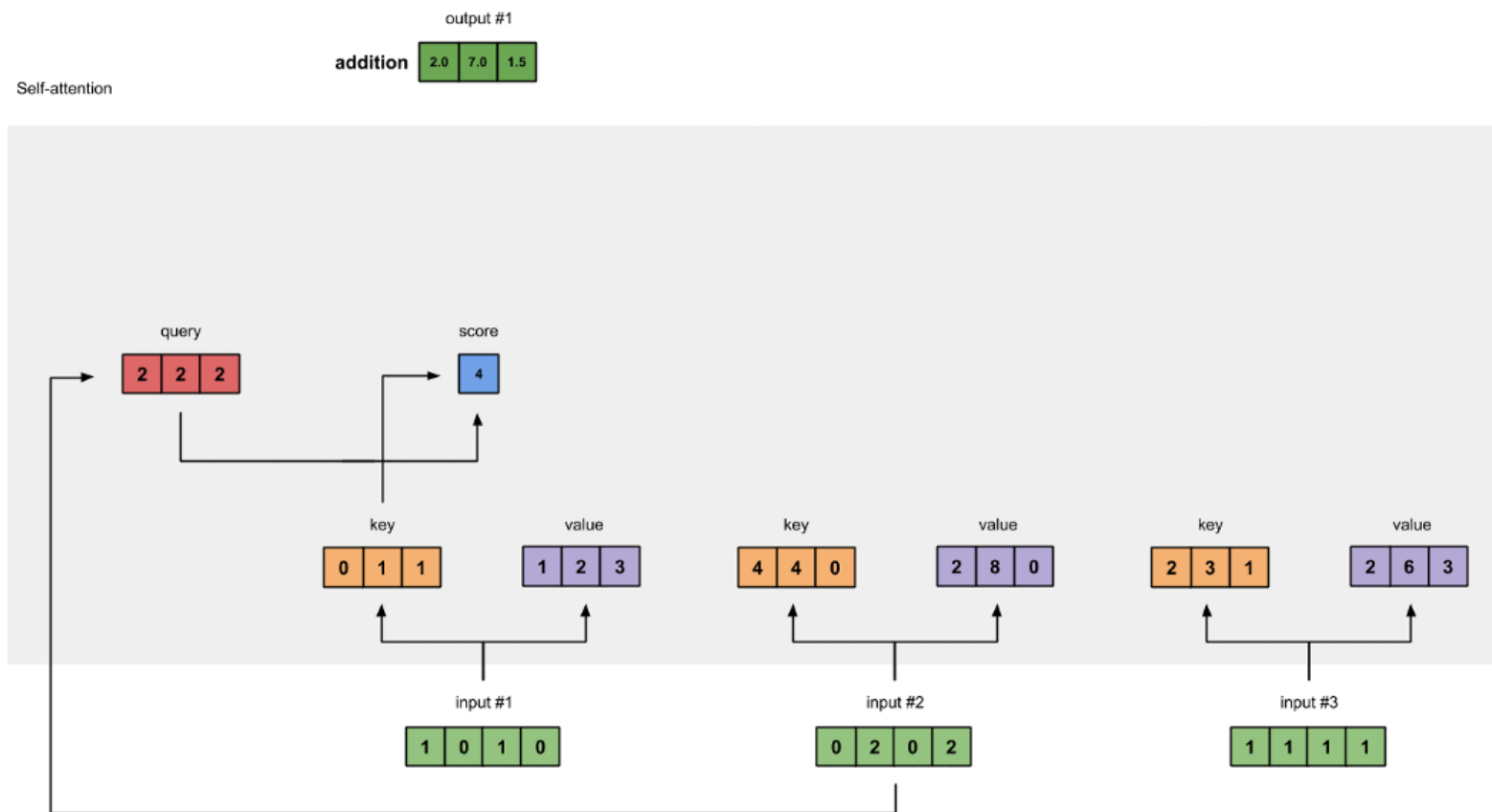
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



Calculate the
Query of the
given input

Transformers

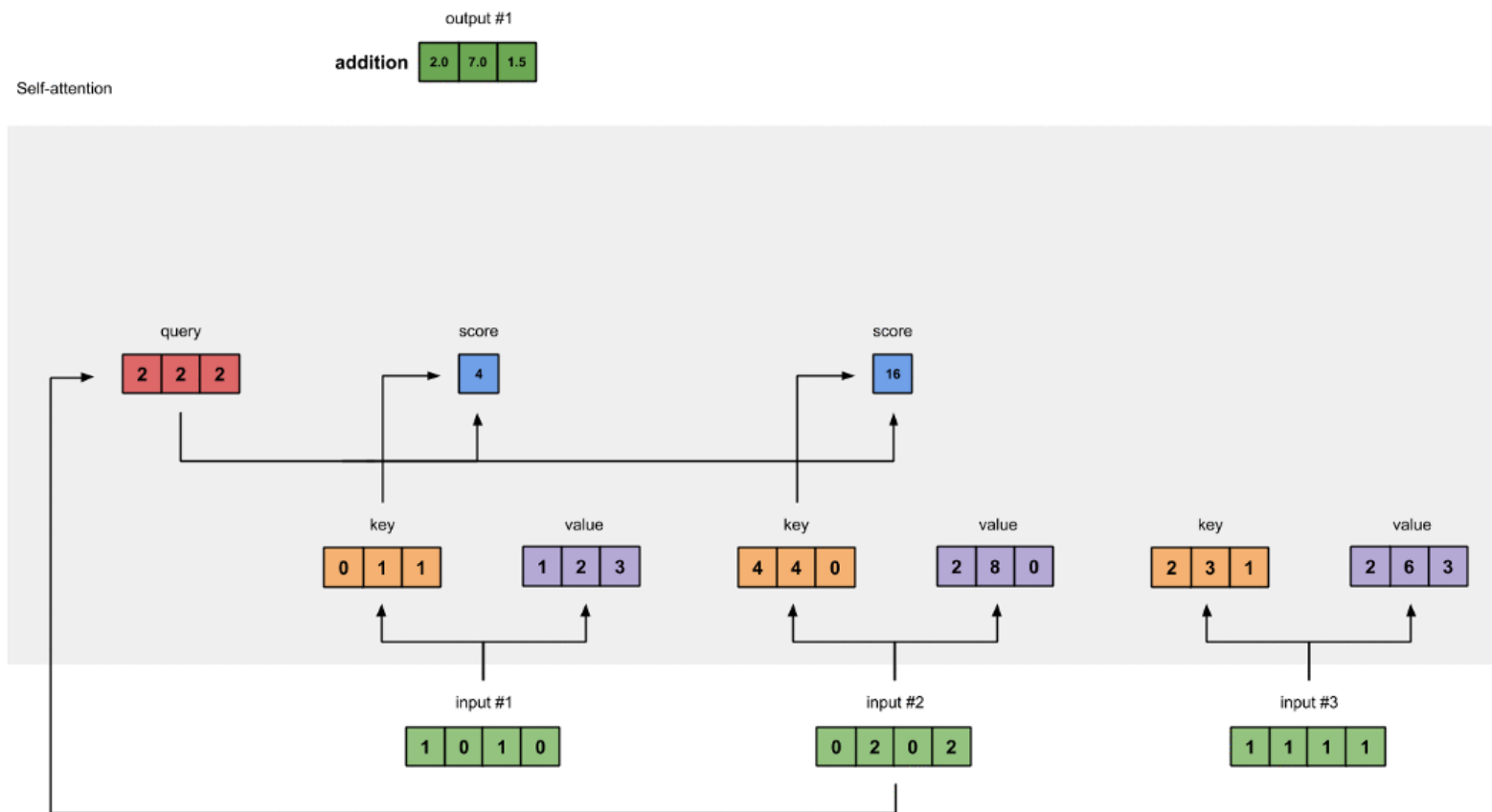
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



Calculate
score as
Query x Key,
over all the
keys in the
sequence

Transformers

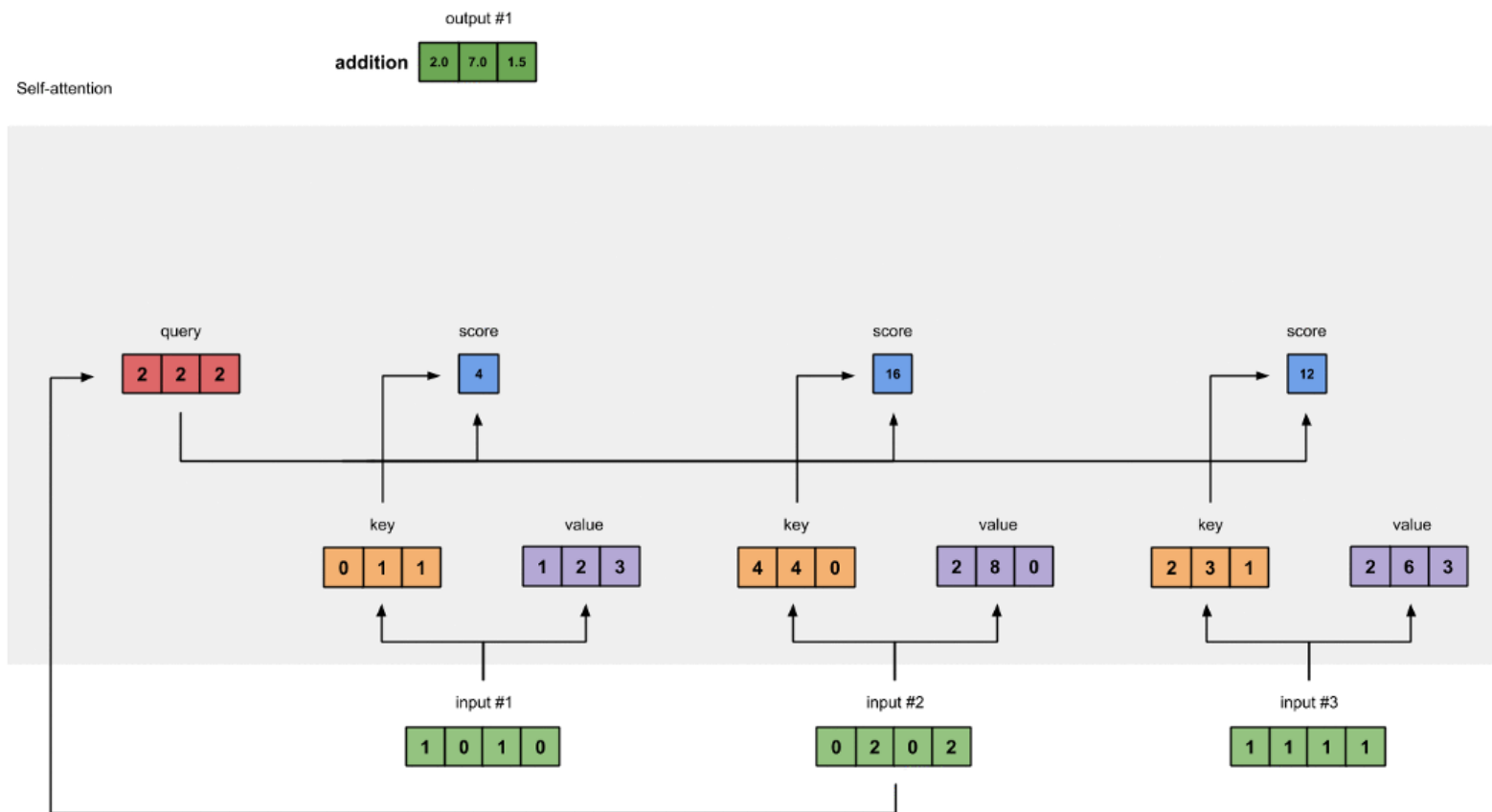
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



Calculate
score as
Query x Key,
over all the
keys in the
sequence

Transformers

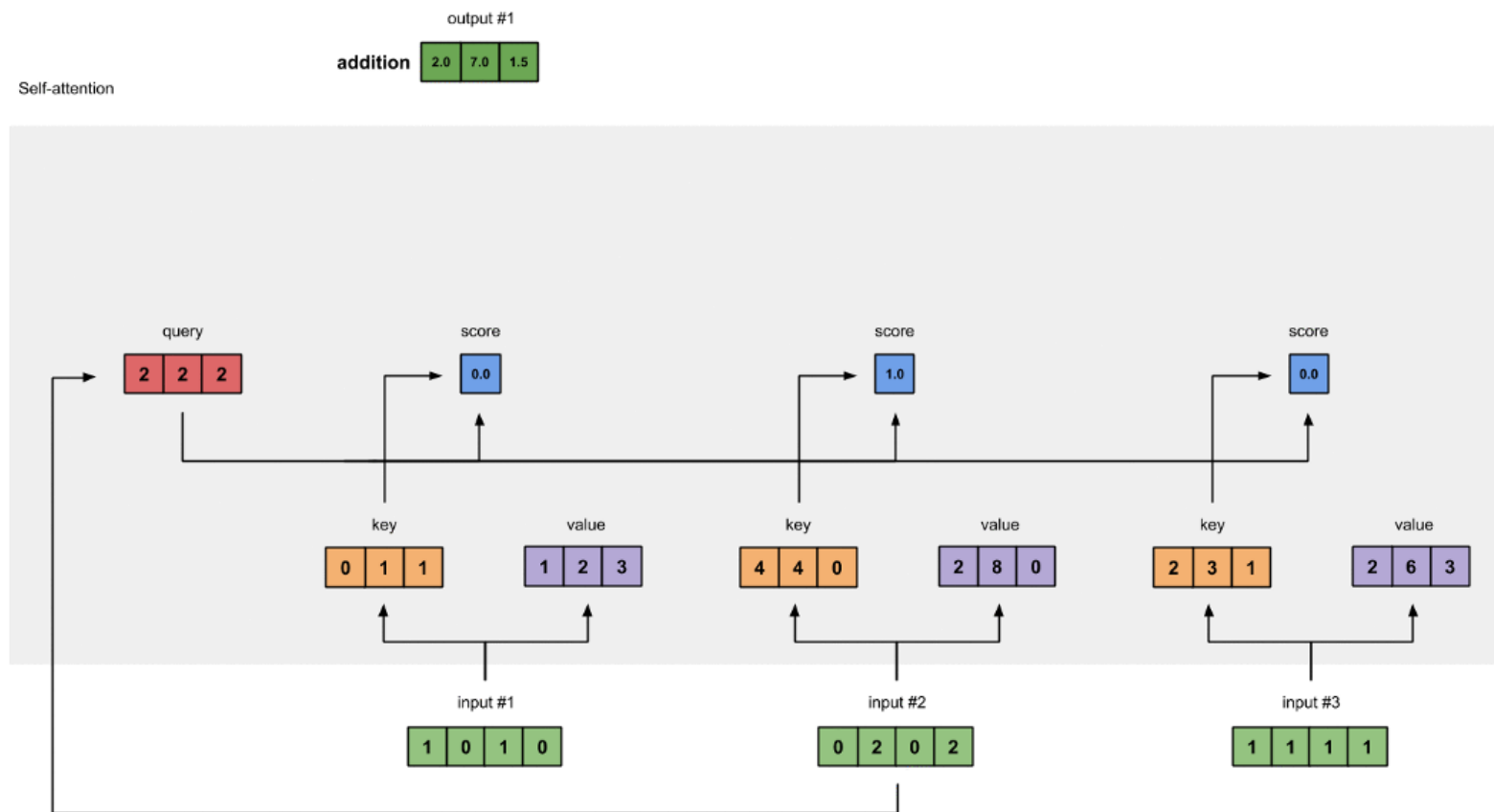
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



Calculate
score as
Query x Key,
over all the
keys in the
sequence

Transformers

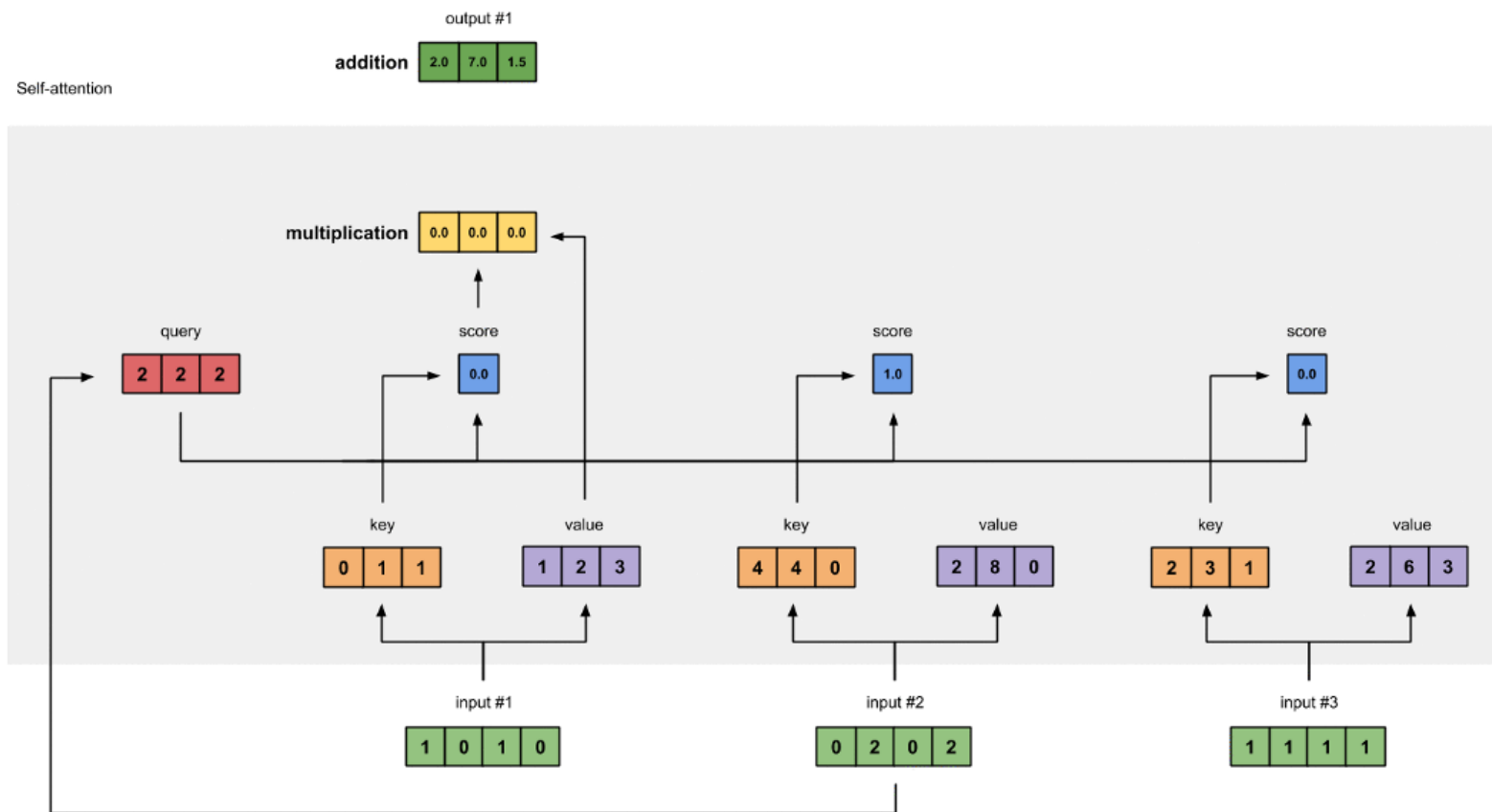
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



Apply
softmax and
transform the
previous
obtained
scores in
probabilities

Transformers

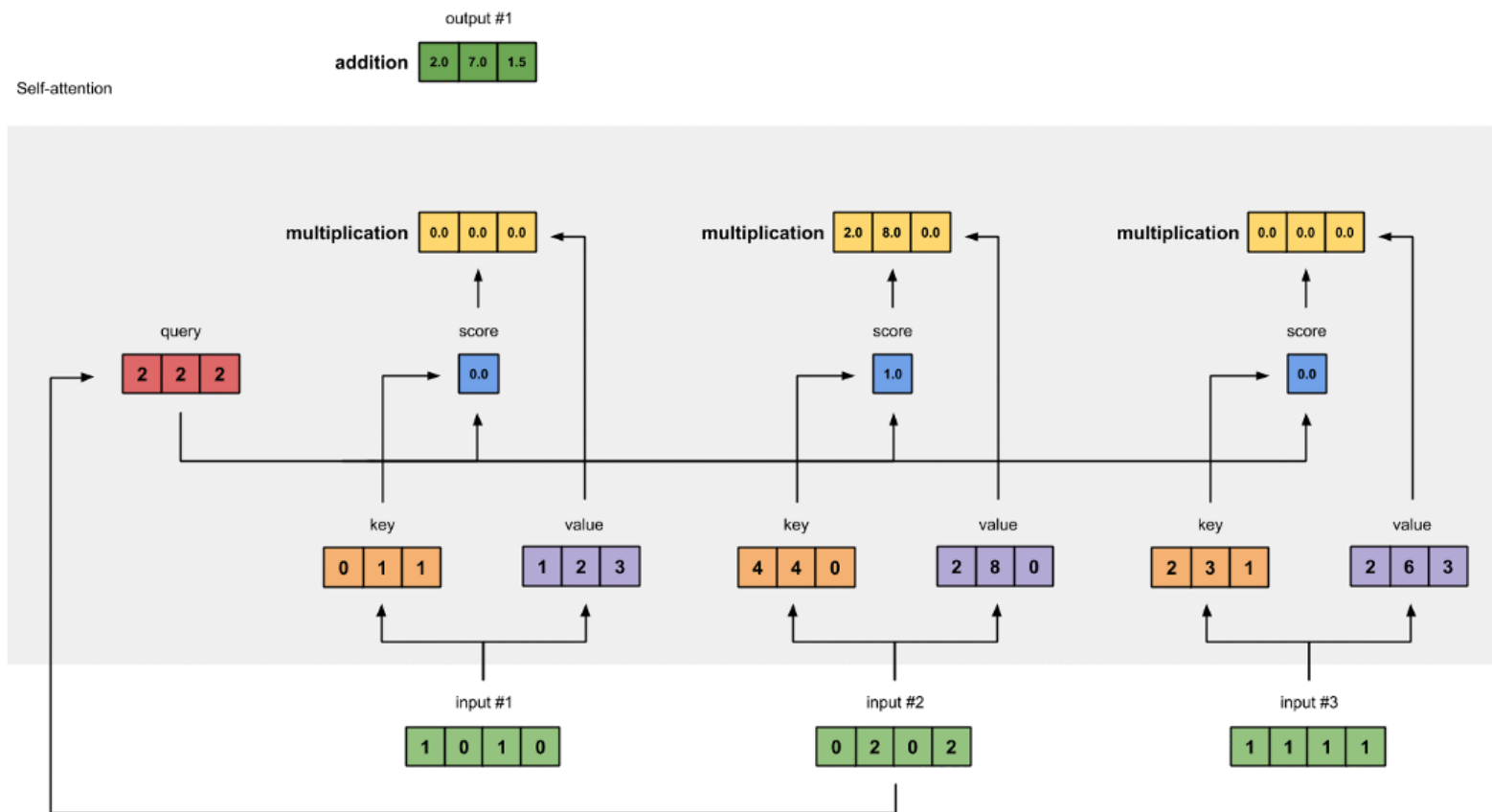
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



Multiply the obtained score for the Value to obtain the attention value associated to the input for the current Query

Transformers

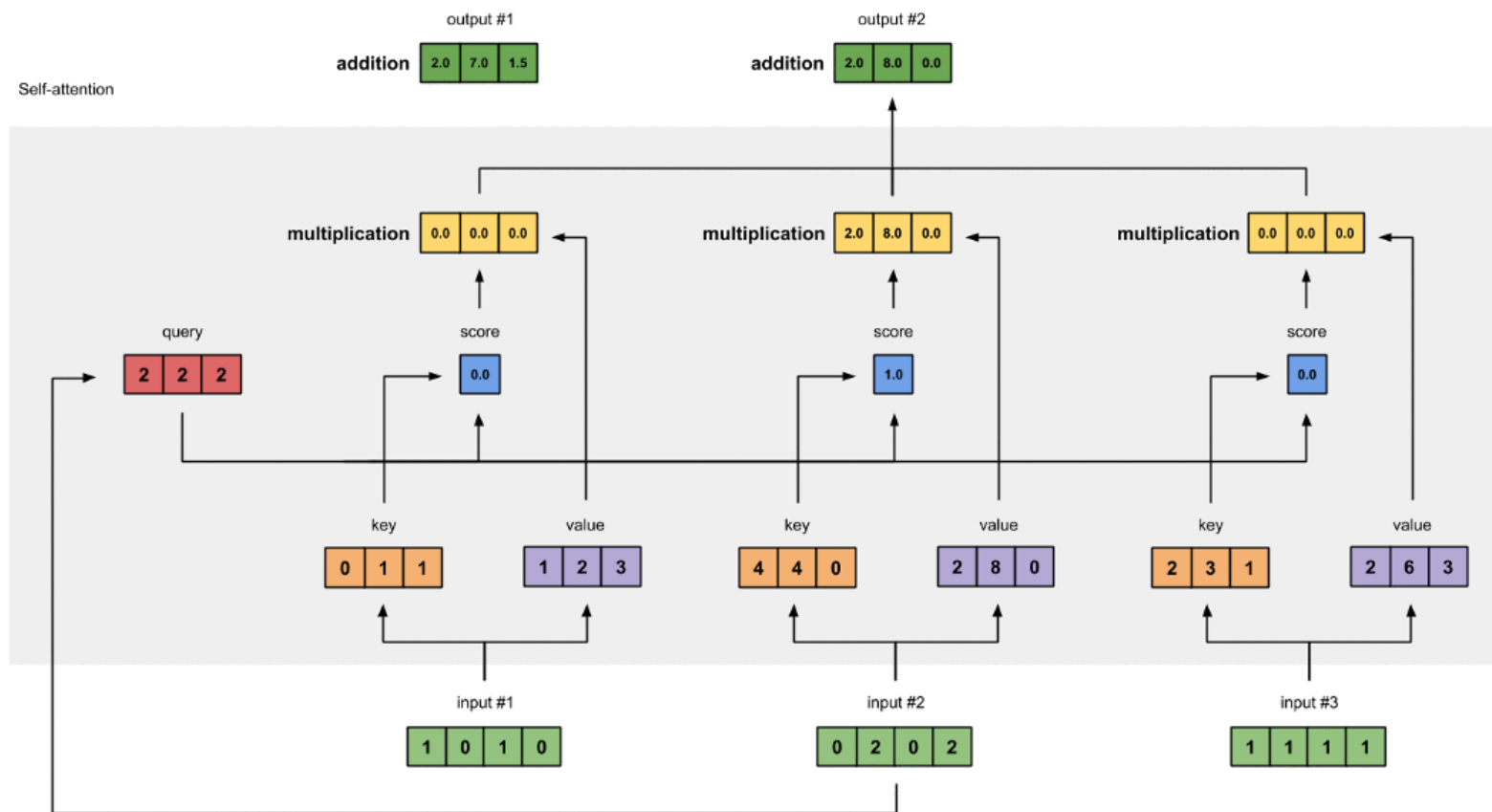
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



Multiply the obtained score for the Value to obtain the attention value associated to the input for the current Query

Transformers

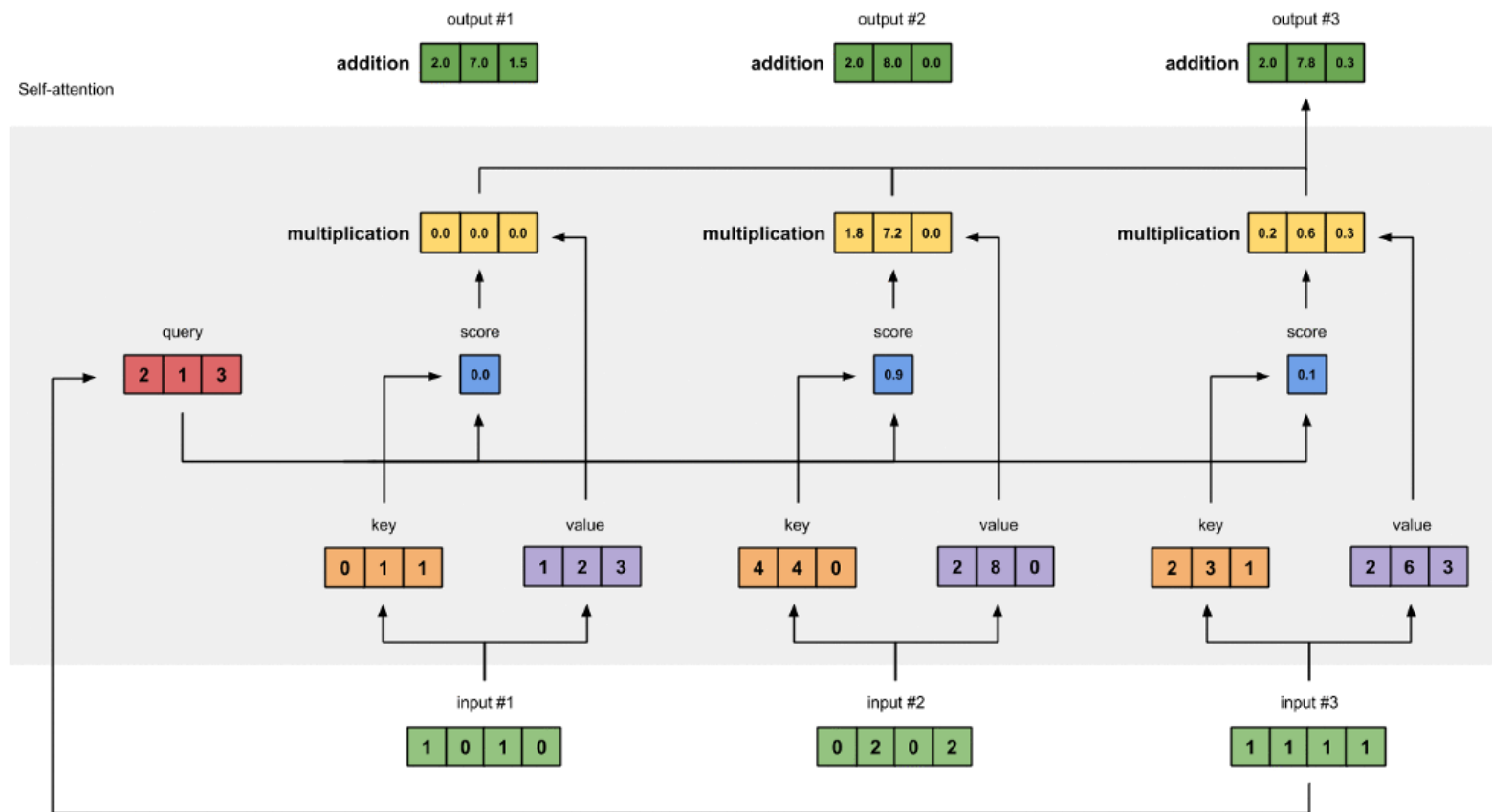
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$



The output is the weighted sum of the obtained attention scores

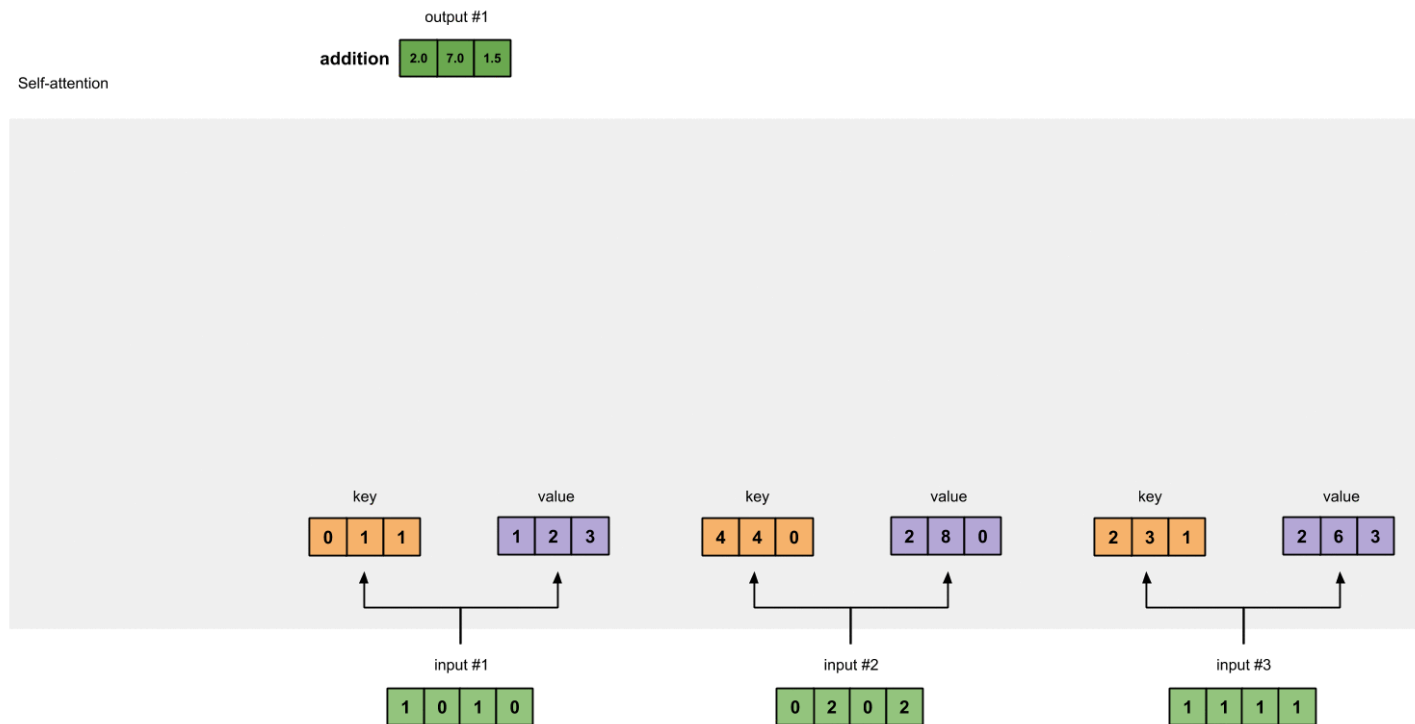
Transformers

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

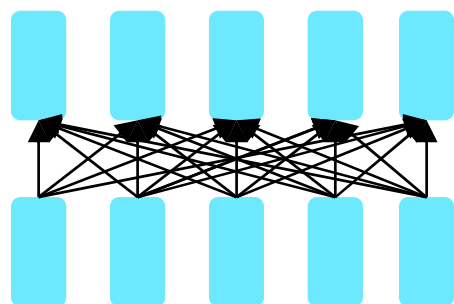


Iterate the process for all inputs in the sequence. It is the self-attention mechanisms and I can see from left-to-right and from right-to-left

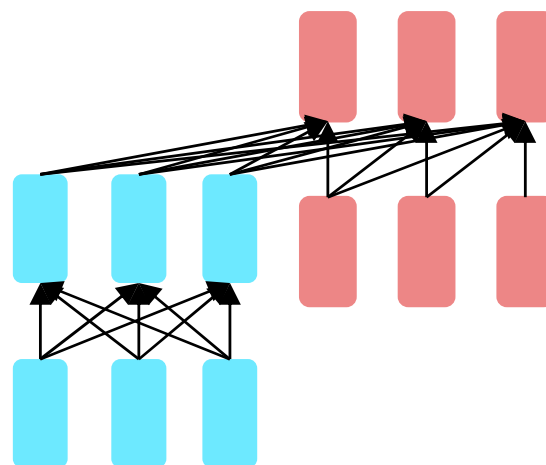
Transformers



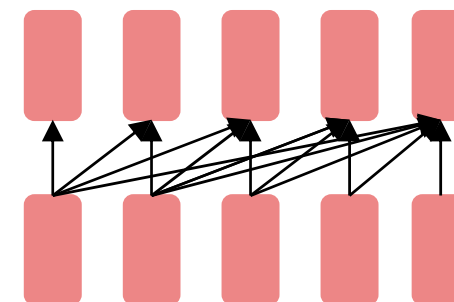
Large Language Models



Encoders
BERT family

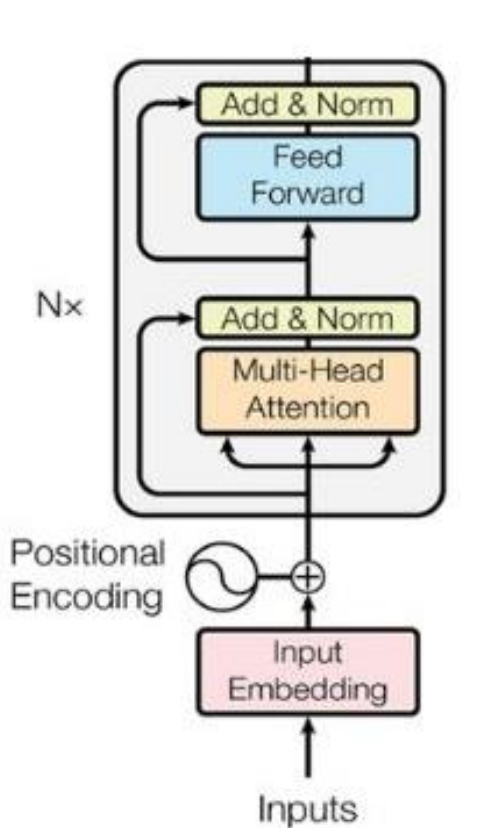


Encoder-decoders
Flan-T5, Whisper

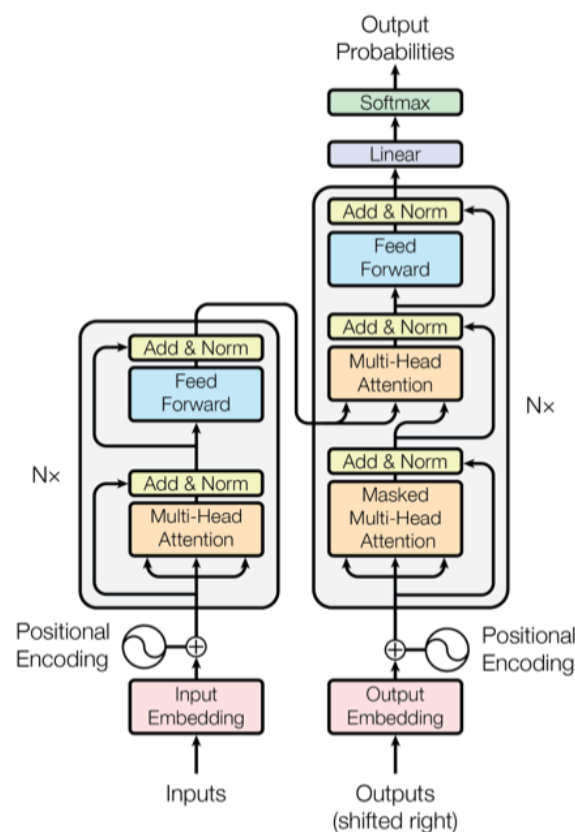


Decoders
GPT, Claude,
Llama, Mixtral

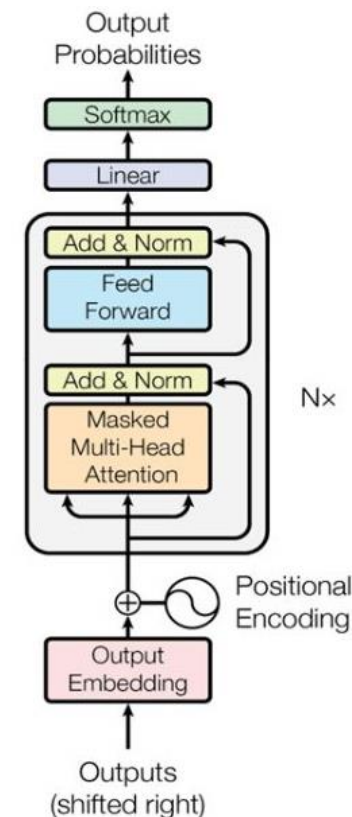
Large Language Models



Encoder-only

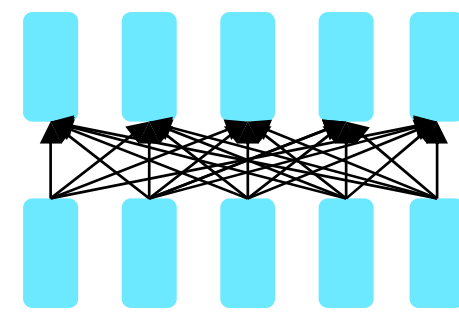


Encoder-decoder



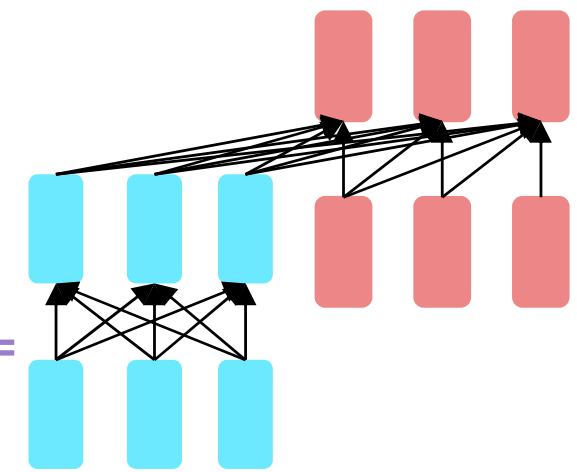
Decoder-only

Encoder-only



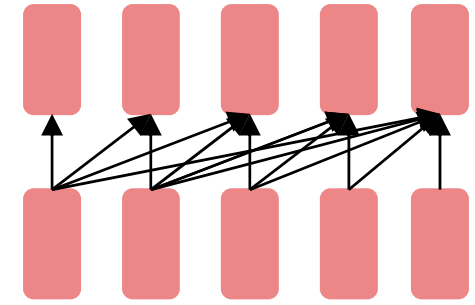
- Masked Language Models (MLMs)
- BERT-like models
- Trained to predict a single masked word inside a given text
 - Fill the gap strategy
- Are usually finetuned (trained with supervised data) to execute specialized tasks s.a. classification

Encoder-Decoders



- Trained to map one sequence to another sequence
 - Machine Translation (mapping across languages)
 - Speech Recognition (mapping from acoustic features to words)

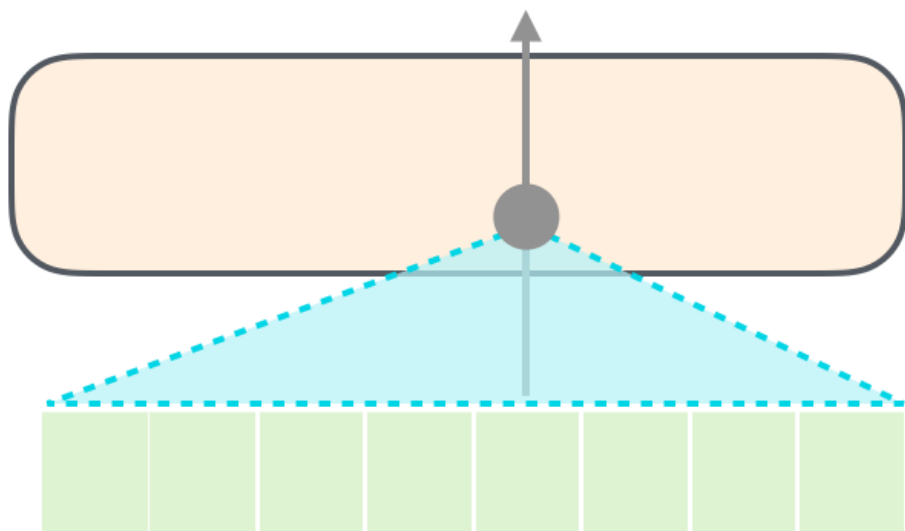
Decoder-only



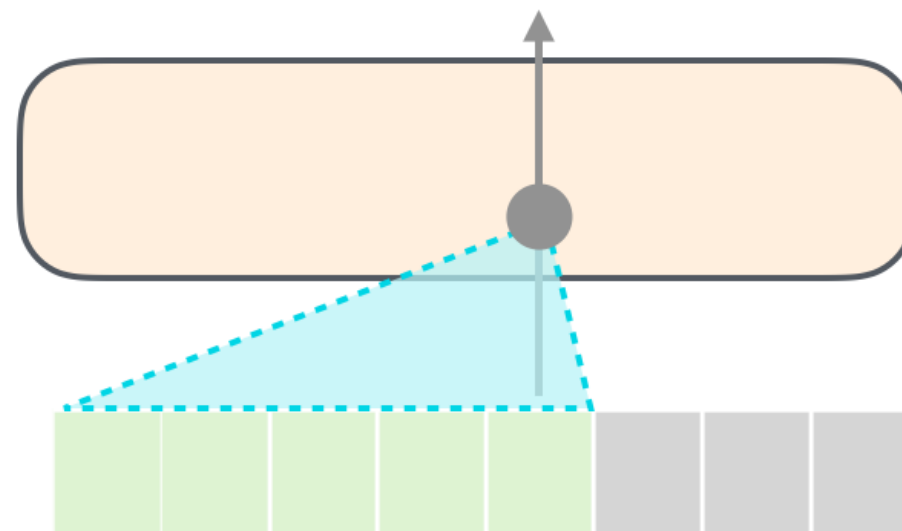
- Causal LLMs, or Autoregressive LLM, or Left-to-right LLMs
- They are generative models, which can predict the next word from a given text
- All the previous tasks can be revised following a generative approach

Decoder-only

Self-Attention

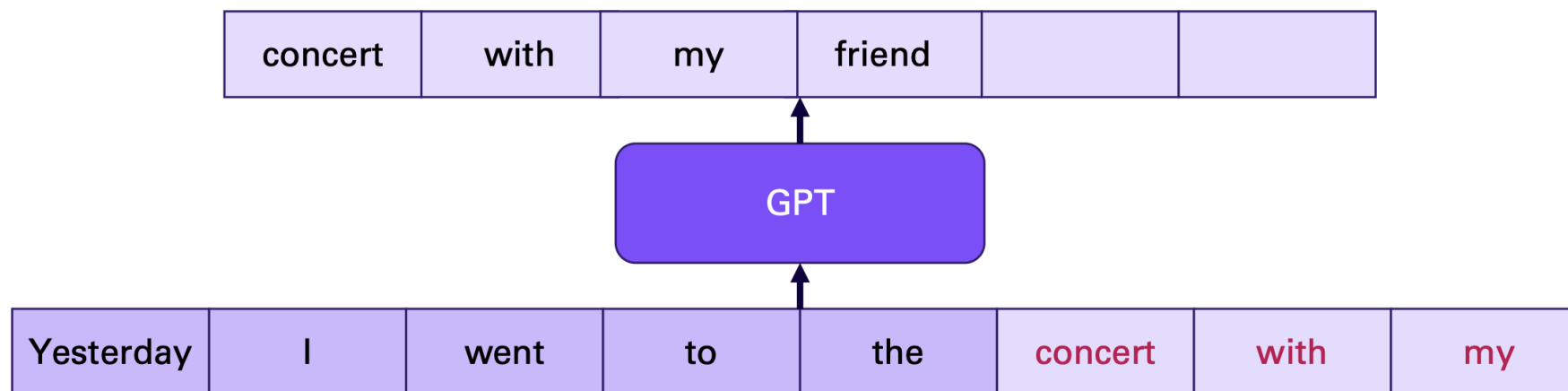


Masked Self-Attention



Decoder-only – Next token prediction

- Masked self-attention + decoder-only
- Training technique to predict next token in a given sequence, the textual content is generated leveraging previous context



Decoder-only – Next token prediction

- Decoder-only models are generative models able to predict and generate newer words from the given input
- This can be simply formalized by the chain rule

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

Decoder-only – Autoregression

- In training phase, the correct token is injected in the model, despite the predicted token which have been generated (teacher forcing strategy)
- In inference phase, the generated token is always injected into the model for subsequent generation (autoregression algorithm)
- If the output depends on previous tokens, I can define an *instruction*, called prompt, to be provided to the model, thus the desired text can be generated

Decoder-only – Autoregression

Algorithm 1 Overview of the autoregressive algorithm

$i \leftarrow 1$

$w_i \sim P(w)$

while $w_i \neq \text{EOS}$ **do**

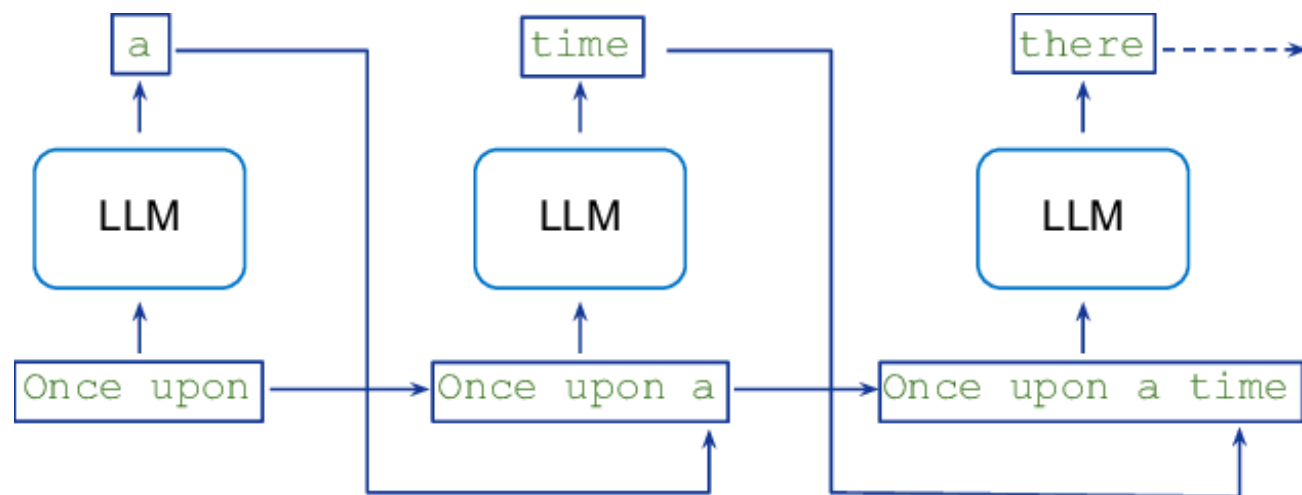
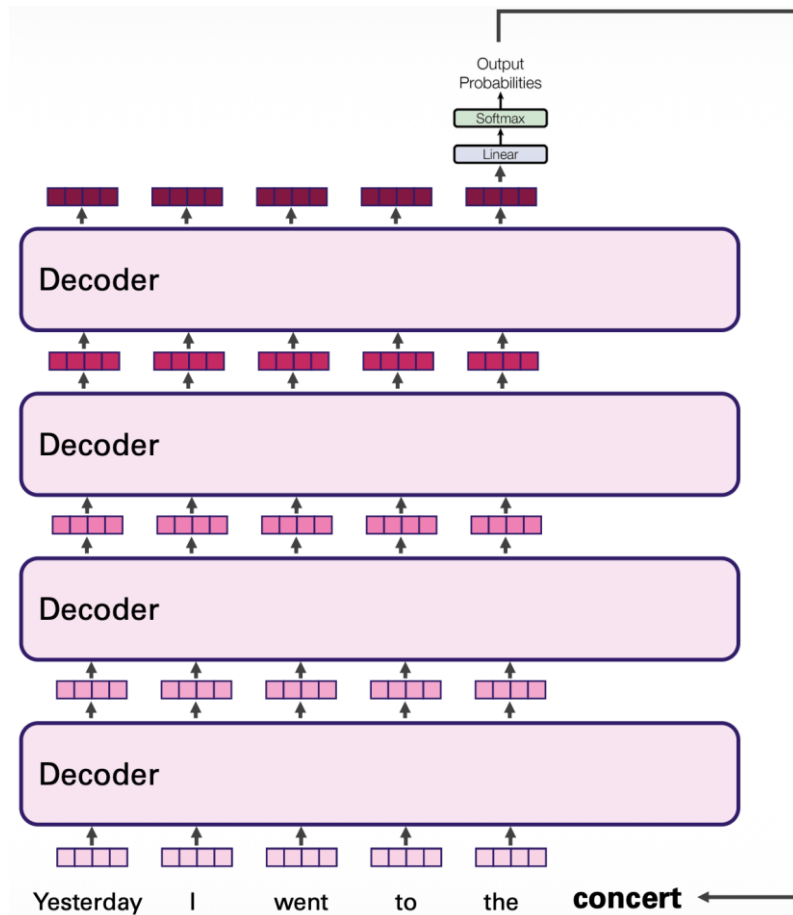
$i \leftarrow i + 1$

$w_i \sim P(w_i | w_{<i})$

end while

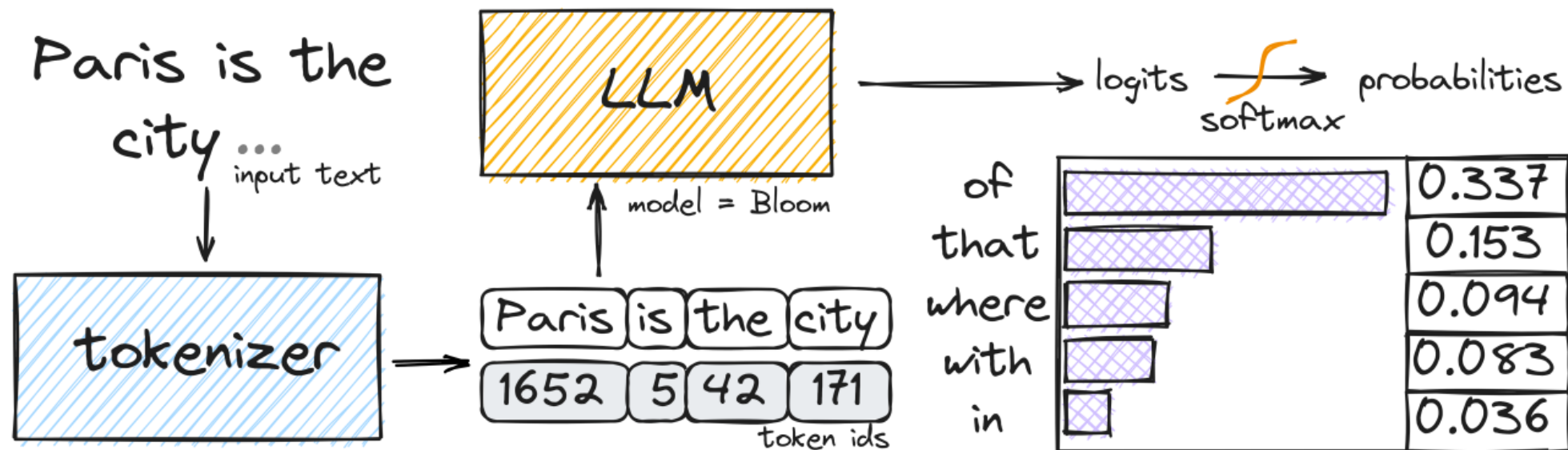
$P(w)$ is the strategy over
which tokens are predicted

Decoder-only – Autoregression



Decoder-only – Decoding

Text input → tokens → LLM → predicted token → predicted text



How do I select the output tokens? [Decoding strategies](#)

Decoder-only – Decoding

- Decoding is the selecting procedure of the next generated token, on the basis of the probabilities provided by the model itself
- Deterministic Decoding
 - Greedy search, beam search
- Stochastic Decoding (sampling)
 - Random sampling, Top-k sampling, Top-p sampling (nucleus sampling), Temperature sampling, contrastive search

Hands on: E1 decoding strategies

Typology of LLMs

- Embedding LLMs
 - Encoder-only models
 - They generate vectorial representations of the given input text
- Generator LLMs
 - Decoder-only models
 - They generate natural language text from the given prompt

Typology of LLMs

- Foundational models
 - Base and general-purpose models
 - Trained over a huge quantity of data to capture a wide linguistic and contextual knowledge
 - Training with self-supervised techniques
- Instruction-tuned models
 - Models trained to follow the input-given instruction
 - Trained over annotated data sets
 - Training with supervised fine-tuning techniques (SFT)

Typology of LLMs

- Multilingual models
 - Base or instruct models with multi-lingual knowledge
 - They can be obtained from scratch or after a language adaptation via fine-tuning
- Multimodal models
 - Base or instruct models
 - Models which can process image, video, audio and textual content in input

Prompting

- Prompt: instruction given as an input to an LLM.
 - It creates a context able to guide the LLM in generating an output compliant with the user's objective
- Prompt Engineering: process to detect the optimal and effective prompt for a given task

Prompting

Hotel Review Completions

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relaxed.

Textual input from which we want a task to be executed

Prompting

Hotel Review Completions

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relaxed. In short our stay was

Prompt for
sentiment
analysis

Prompting

Hotel Review Completions

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relaxed. In short our stay was

... not a pleasant one. The staff at the front desk were not welcoming or friendly, and seemed disinterested in providing good customer service.

... uncomfortable and not worth the price we paid. We will not be returning to this hotel.

Negative text which affects the sentiment.

Prompting

- Prompts can be used for a huge variety of tasks
- Summarization
- Translation
- (Fine-grained) sentiment analysis
- ...

Templates

- We can rely on templates and reuse them for the same task, instead of re-write it everytime

Basic Prompt Templates

Summarization	<code>{input} ; tldr;</code>
Translation	<code>{input} ; translate to French:</code>
Sentiment	<code>{input}; Overall, it was</code>
Fine-Grained-Sentiment	<code>{input}; What aspects were important in this review?</code>

Templates

- Prompts and templates should be designed in a non ambiguous manner, thus the desired output can be achieved.

A prompt consisting of a review plus an incomplete statement

Human: Do you think that “input” has negative or positive sentiment?

Choices:

(P) Positive

(N) Negative

Assistant: I believe the best answer is: (

Templates

LLM Outputs for Basic Prompts

Original Review (\$INPUT)	Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life.
Sentiment	Prompt: \$INPUT + In short, our stay was Output: not enjoyable
Fine-grained Sentiment	Prompt: \$INPUT + These aspects were important to the reviewer: Output: 1. Poor service 2. Unpleasant location 3. Noisy and busy area
Summarization	Prompt: \$INPUT + tl;dr Output: I had a bad experience with the hotel's service and the location was loud and busy.
Translation	Prompt: \$INPUT + Translate this to French Output: Je n'ai pas aimé le service qui m'a été offert lorsque je suis entré dans l'hôtel. Je n'ai également pas aimé la zone dans laquelle se trouvait l'hôtel. Trop de bruit et d'événements pour que je me sente détendu et loin de la vie citadine.

Prompting with templates process

1. For a given task, design a suitable and specific template with a free input field.
2. Given the input and the template, the textual input is used to initialize the template (which now is the actual prompt) and given to the selected LLM.
3. With autoregressive decoding, the output token with the predicted answer are generated.
4. The output can be directly used as output or the answer can be extracted and/or refined from the generated content.

Few-shot prompting

- Improve a prompt by inclusion of labeled examples.
- On the contrary, zero-shots prompting uses prompts without any labeled samples.
- We can have zero-, one-, *n-shot prompting* or, in general, few-shot prompting

Few-shot prompting

Definition: This task is about writing a correct answer for the reading comprehension task. Based on the information provided in a given passage, you should identify the shortest continuous text span from the passage that serves as an answer to the given question. Avoid answers that are incorrect or provides incomplete justification for the question.

Passage: Beyoncé Giselle Knowles-Carter (born September 4, 1981) is an American singer, songwriter, record producer and actress. Born and raised in Houston, Texas, she performed in various singing and dancing competitions as a child, and rose to fame in the late 1990s as lead singer of R&B girl-group Destiny's Child. Managed by her father, Mathew Knowles, the group became one of the world's best-selling girl groups of all time. Their hiatus saw the release of Beyoncé's debut album, *Dangerously in Love* (2003), which established her as a solo artist worldwide, earned five Grammy Awards and featured the Billboard Hot 100 number-one singles "Crazy in Love" and "Baby Boy".

Examples:

Q: In what city and state did Beyoncé grow up?

A: Houston, Texas

Q: What areas did Beyoncé compete in when she was growing up?

A: singing and dancing

Q: When did Beyoncé release *Dangerously in Love*?

A: 2003

Q: When did Beyoncé start becoming popular?

A:

Few-shot prompting

How many examples?

- Non too many
- Higher increase in performance are obtained by first sample and decrease with the subsequents.
- Are useful to demonstrate the task to the LLM and the expected answer format.
- Samples can be both selected manually or automatically

In-Context Learning

- All prompts can be seen as a learning process
 - The more LLM reads the prompt, the more its generation capabilities improves, mainly thanks to the given contextual information provided.
- This learning capability of generative LLM through prompt is referred to as in-context learning.

Chain-of-Thought Prompting

- Chain-of-Thought prompting (COT) is a technique to improve performances over difficult reasoning tasks over which LLMs tend to fail.
- People usually solve complex tasks by dividing it in simpler and smaller steps (divide et impera approach).
- In COT, the instruction in the prompt forces the LLM to act in the same way.

Chain-of-Thought Prompting

- Each sample in few-shot prompting is enriched with textual content which explains some reasoning steps, thus generating a logical demonstration.
- The objective is to force the LLM in producing analogous reasoning steps for the problem to be solved and in generating the correct answer as output.

Chain-of-Thought Prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

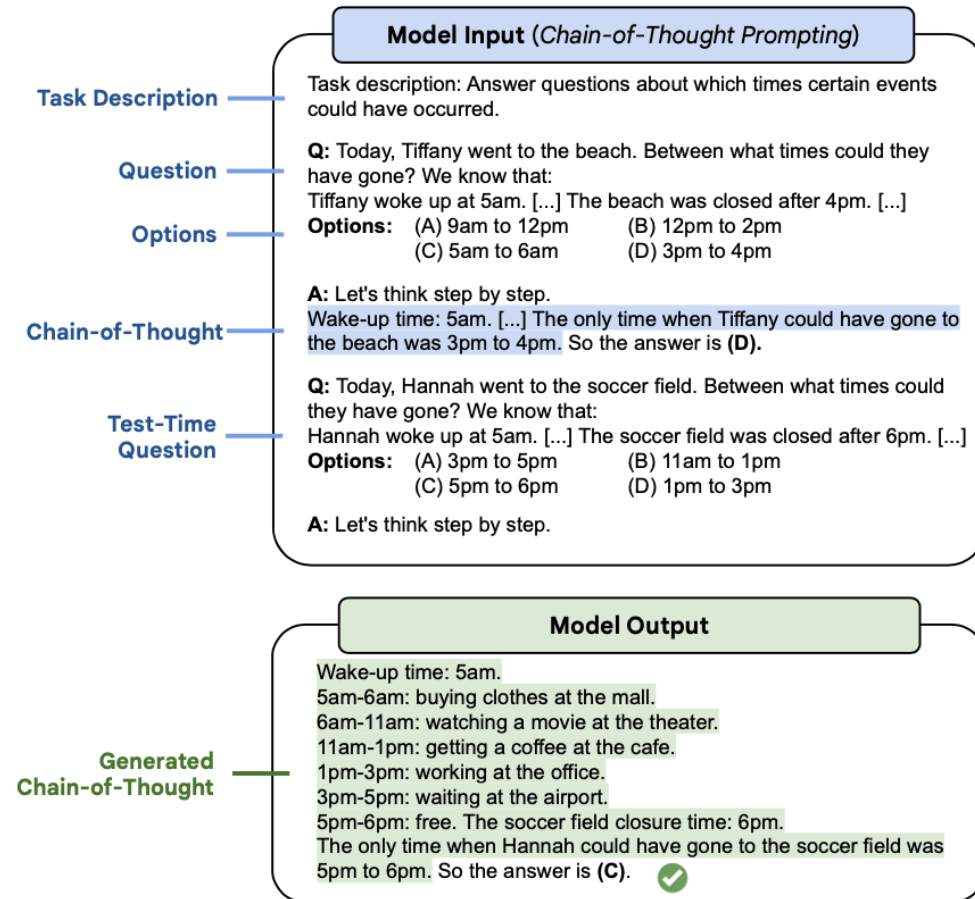
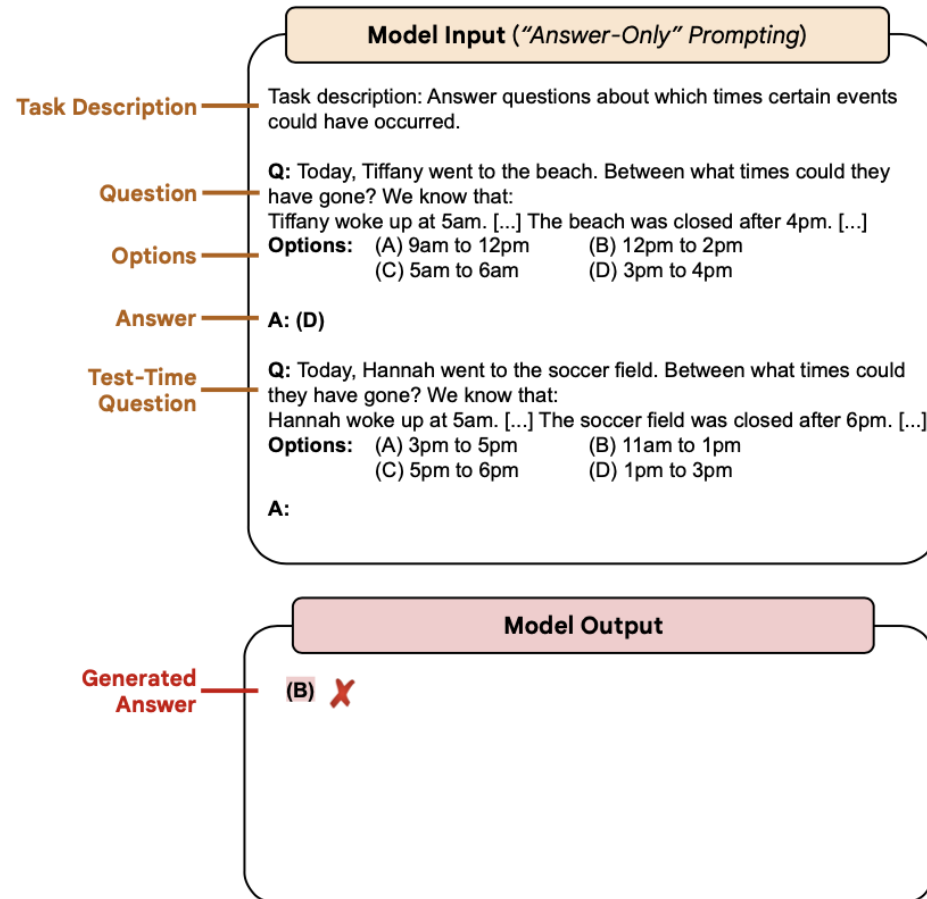
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-Thought Prompting



open-source models @ HuggingFace



Libraries

- Model usage and inference

 [HuggingFace](#) (Transformers)

 [vLLM](#)

 [Ollama](#)

- Complex pipelines

 [LangChain](#)

 [LlamaIndex](#)

- Training

 [Unsloth](#)

 HuggingFace (PEFT, Accelerate, TRL, bitsandbytes)

Bibliography

- Jurafsky D. and Martin J. H., Speech and Language Processing (3rd ed. draft),
<https://web.stanford.edu/~jurafsky/slp3/>

Hands on: E2 instruct inference